

Programme	Advanced Diploma in Information and Communication Technology
Module	NADV744 – Advanced Development Systems
Examiner	Melvin Kisten
Due date	24 August 2026
GitHub repository	https://github.com/Maje107/ADVANCED-DEVELOPMENT-SOFTWARE-PROJECT

Group members

Full name	Student number
Kamogelo Bantsheng	202311455
Gadifele Serame	202219593
Gomolemo Precious Walaza	202200387
Tshepang Regality Maje	202340368
Ofentse Batlang	202300977
Luyanda Nhlapho	202213238

Table of Contents

1. Problem definition	3
1.1 Problem Statement.....	3
1.2 Why it matters in ICT	3
1.3 Plan and resources	3
2. System design	3
2.1 Architecture Overview.....	3
2.2 Layered Architecture	4
2.3 Data Flow.....	5
2.4 API endpoints	5
2.5 Tools & Technologies	6
3. Implementation	6
3.1 Project structure(simplified).....	6
3.2 Sample code (Dashboard views.py)	7
3.3 GitHub	7
3.4 Error Handling Example	7
4. Testing and evaluation	8
4.1 Automated unit tests	8
4.2 Unit Testing	8
4.3 Performance and scalability notes.....	8
4.4 Api Testing	9
5. Report	9
5.1 Background and Context.....	9
5.2 Methods	10
5.3 Results and Discussion.....	10
5.4 Limitations	10
5.5 Future improvements	10
6. CODE & DEMO	10
6.1 How to run the System	12
7. Conclusion	13

1. Problem definition

1.1 Problem Statement

Sol Plaatje University (SPU) currently does not have a centralised digital system for booking and managing campus venues. Venue bookings are managed manually through emails, paper forms, and phone calls. Rooms, laboratories, lecture halls, tutorial rooms, and event spaces across the university are therefore allocated on an ad-hoc basis.

This manual process can lead to scheduling conflicts, double bookings, missing or unrecorded booking information, limited visibility of venue availability, and a high administrative workload. Students, lecturers, and student leaders do not have real-time access to venue availability, while administrators have limited ability to track approvals, cancellations, and booking history. A centralised campus booking system is therefore needed to improve the efficiency, accuracy, and management of venue bookings at SPU.

1.2 Why it matters in ICT

The system applies core ICT competencies, including REST API design, service-oriented architecture, role-based access control (RBAC), relational database modelling, and token-based authentication using JSON Web Tokens (JWT). It also incorporates software testing to ensure the reliability and security of the system. By automating the manual venue-booking process, the system improves efficiency, transparency, and accountability across the campus. It demonstrates how a well-engineered digital service can replace inefficient manual administrative processes with a centralized, secure, and reliable solution.

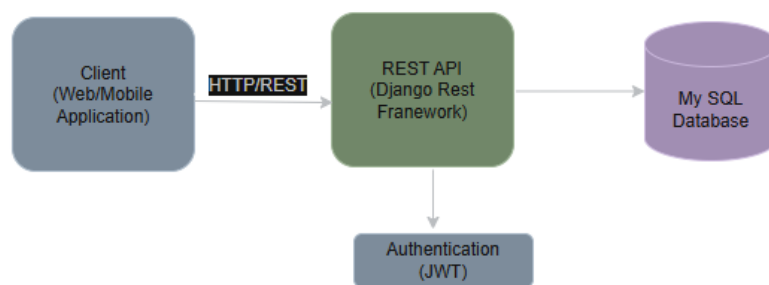
1.3 Plan and resources

Resource	Description	Purpose
Hardware	Laptop, Internet	Development and Testing
Software	Python 3,13, Django 5.x, DRF, MySQL, Postman, VS Code	Build, test, and manage the system
Methodology	Agile (Sprint-based)	Iterative development
Team Roles	Developer, Tester, UI Developer, Documenter, DevOps	Division of responsibilities
Timeline	4 weeks	Development Cycle

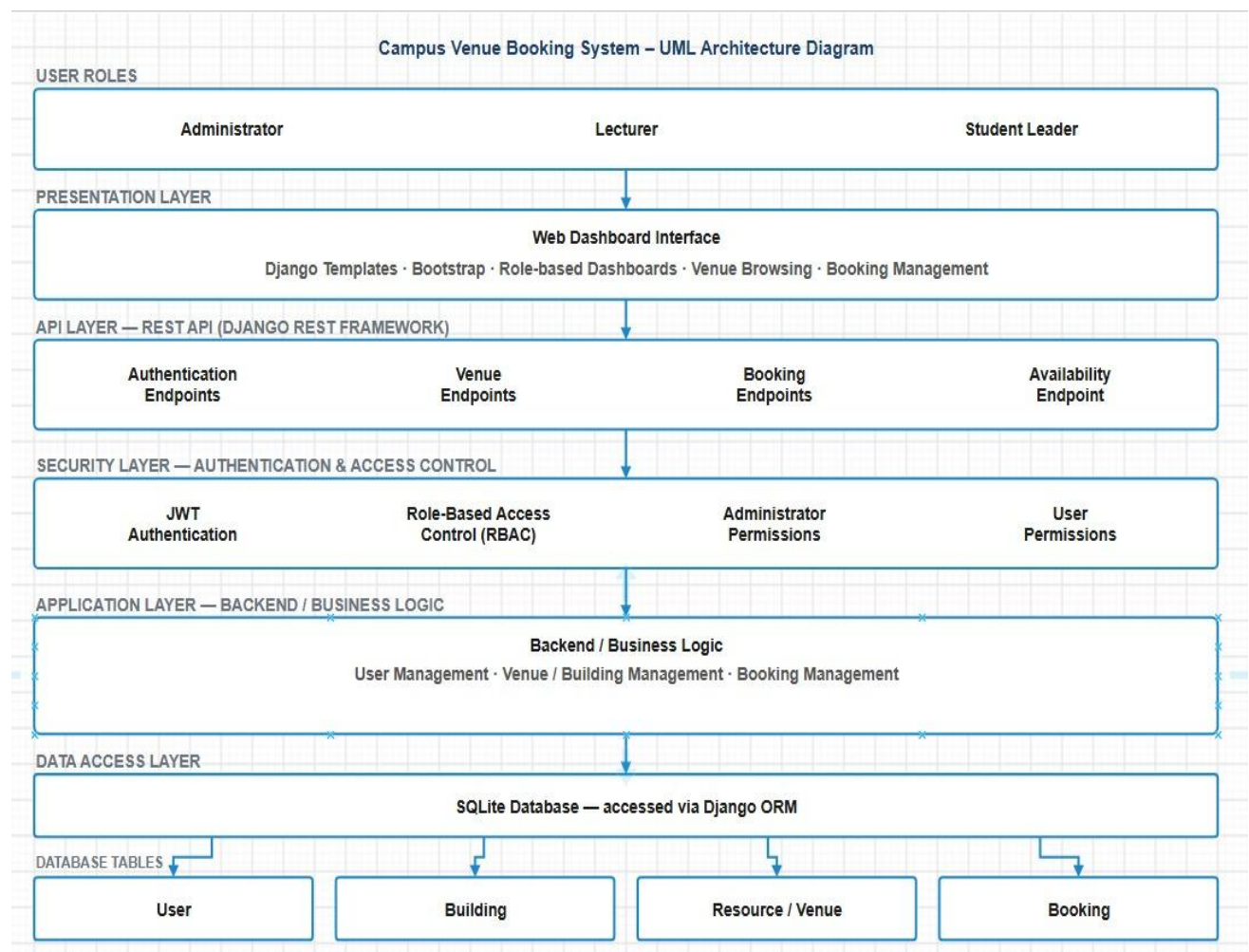
2. System design

2.1 Architecture Overview

The system follows a **layered, service-oriented Django architecture** with three main layers: **presentation, application/service, and data**. The presentation layer provides a REST API and web dashboard, while the application layer handles business logic. The data layer uses a relational database to store system information. Both interfaces share the same business logic and database, avoiding duplication and maintaining consistency.



2.2 Layered Architecture

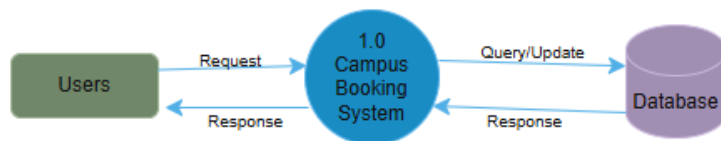


The diagram above shows the system's layered architecture, tracing a request from user to database. The three user roles — Administrator, Lecturer, and Student Leader — all interact through the Presentation Layer, a Django-templated web dashboard that handles venue browsing and booking management. Requests then pass into the API Layer, where Django REST Framework exposes dedicated endpoints for authentication, venues, bookings, and availability. Every request is validated by the Security Layer, which enforces JWT authentication and role-based access control (RBAC) before any data operation is permitted. Validated requests reach the Application Layer, where the core business logic — user management, venue/building management, and booking management — is executed. Finally, the Data Access Layer persists and retrieves data through the Django ORM against a SQLite database, structured into four core tables: User, Building, Resource/Venue, and Booking.

2.3 Data Flow

Booking request flow:

1. The user selects a venue and submits a booking request.
2. The system validates the booking details and operating hours.
3. It checks for conflicts with existing approved bookings.
4. If no conflict exists, the booking is created with **pending** status.
5. An administrator reviews and approves or declines the request.
6. On approval, the system performs a final conflict check before updating the status.
7. The user's booking list displays the updated **approved/declined** status.



2.4 API endpoints

Authentication endpoints (/api/auth/):

Method	Endpoint	Description	Access
POST	/api/auth/register	Register a Lecturer or Student Leader	Public
POST	/api/auth/login	Obtain JWT access and refresh tokens	Public
POST	/api/auth/token/refresh	Refresh an expired access token	Public
GET	/api/auth/me	Retrieve the authenticated user's profile	JWT Bearer
POST	/api/auth/forgot password	Generate a password reset token	Public
POST	/api/auth/reset password	Reset password using a valid token	Public

Venue and building endpoints (/api/resources/):

Method	Endpoint	Description	Access
GET	/api/resources/buildings	List all seeded buildings	Public
GET	/api/resources/venues	List/filter venues (building, type, capacity, search)	Public
POST	/api/resources/venues/create	Create a new venue	Admin only
GET	/api/resources/venues/<id>	Venue detail with building info	Public

Method	Endpoint	Description	Access
PUT/PATCH	/api/resources/venues/<id>/update	Update venue details	Admin only
DELETE	/api/resources/venues/<id>	Delete a venue	Admin only
GET	/api/resources/venues/<id>/availability	Calculate booked and free time slots for a date	Public

Booking endpoints (/api/resources/bookings):

Method	Endpoint	Description	Access
GET	/api/resources/bookings	List bookings (admin sees all; users see own)	JWT Bearer
POST	/api/resources/bookings	Submit a new booking request (conflict checked)	JWT Bearer
GET	/api/resources/bookings/<id>	Retrieve a specific booking	JWT Bearer
PUT	/api/resources/bookings/<id>/cancel	Cancel own booking	Booker / Admin
PUT/PATCH	/api/resources/bookings/<id>/status	Approve or decline a booking	Admin only

API security

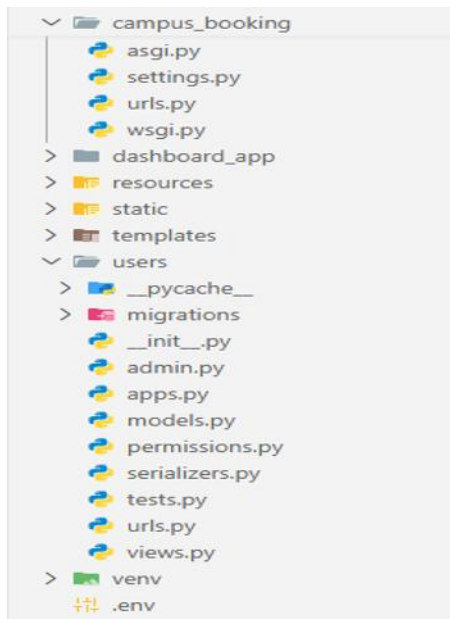
All authenticated API endpoints require a valid JWT access token in the Authorization header (Bearer <token>). Role enforcement is implemented as custom DRF permission classes (IsAdmin, IsBooker) applied at the view level. The web dashboard separately uses Django session authentication with server-side role checks on every request, so hiding a menu item is never the only protection on a route.

2.5 Tools & Technologies

- Backend: Django 5.x (DRF)
- Database: MySQL 8.0
- API Testing: Django Test Case (unit test)
- Version Control: Git and GitHub
- Frontend: Django templates + Bootstrap 5 + custom SPU styling
- Environment: VS Code
- OS: Windows 11

3. Implementation

3.1 Project structure(simplified)



3.2 Sample code (Dashboard views.py)

```
def dashboard_signup(request):
    selected_role = request.session.get('selected_role', 'student_leader')
    if request.method == 'POST':
        full_name = request.POST.get('full_name', '').strip()
        email = request.POST.get('email', '').strip().lower()
        password = request.POST.get('password', '')
        password_confirm = request.POST.get('password_confirm', '')
        role = request.POST.get('role', selected_role)
        student_number = request.POST.get('student_number', '').strip() or None
        phone_number = request.POST.get('phone_number', '').strip()
        leadership_role = request.POST.get('leadership_role', '')

        if not all([full_name, email, password, password_confirm]):
            messages.error(request, 'All required fields must be completed.')
        elif password != password_confirm:
            messages.error(request, 'Passwords do not match. Please re-enter.')
        elif len(password) < 8:
            messages.error(request, 'Password must be at least 8 characters long.')
        elif role == 'admin':
            messages.error(request, 'Administrator registration is not permitted. Please contact IT S
        elif User.objects.filter(email=email).exists():
            messages.error(request, 'An account with this email address is already registered.')
        elif student_number and User.objects.filter(student_number=student_number).exists():
            messages.error(request, 'An account with this student number is already registered.')
        else:
            try:
                user = User.objects.create_user(
                    full_name=full_name,
                    email=email,
```

3.3 GitHub

<https://github.com/Maje107/ADVANCED-DEVELOPMENT-SOFTWARE-PROJECT>

3.4 Error Handling Example

```

if 'unique' in str(e).lower() or 'Duplicate' in str(e):
    return Response(
        {'detail': 'Email or student number already registered.'},
        status=status.HTTP_409_CONFLICT,
    )
return Response({'detail': f'Registration failed: {str(e)}'}, status=status.HTTP_400_BAD_REQUEST)

```

4. Testing and evaluation

4.1 Automated unit tests

The project ships with an automated test suite covering both apps. Run with `python manage.py test` (Pytest)

```

(venv) PS C:\Users\Admin\Downloads\campus_1234 (1)\campus_1234\campus_1234> python manage.py test
Found 9 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
.....
Ran 9 tests in 27.997s

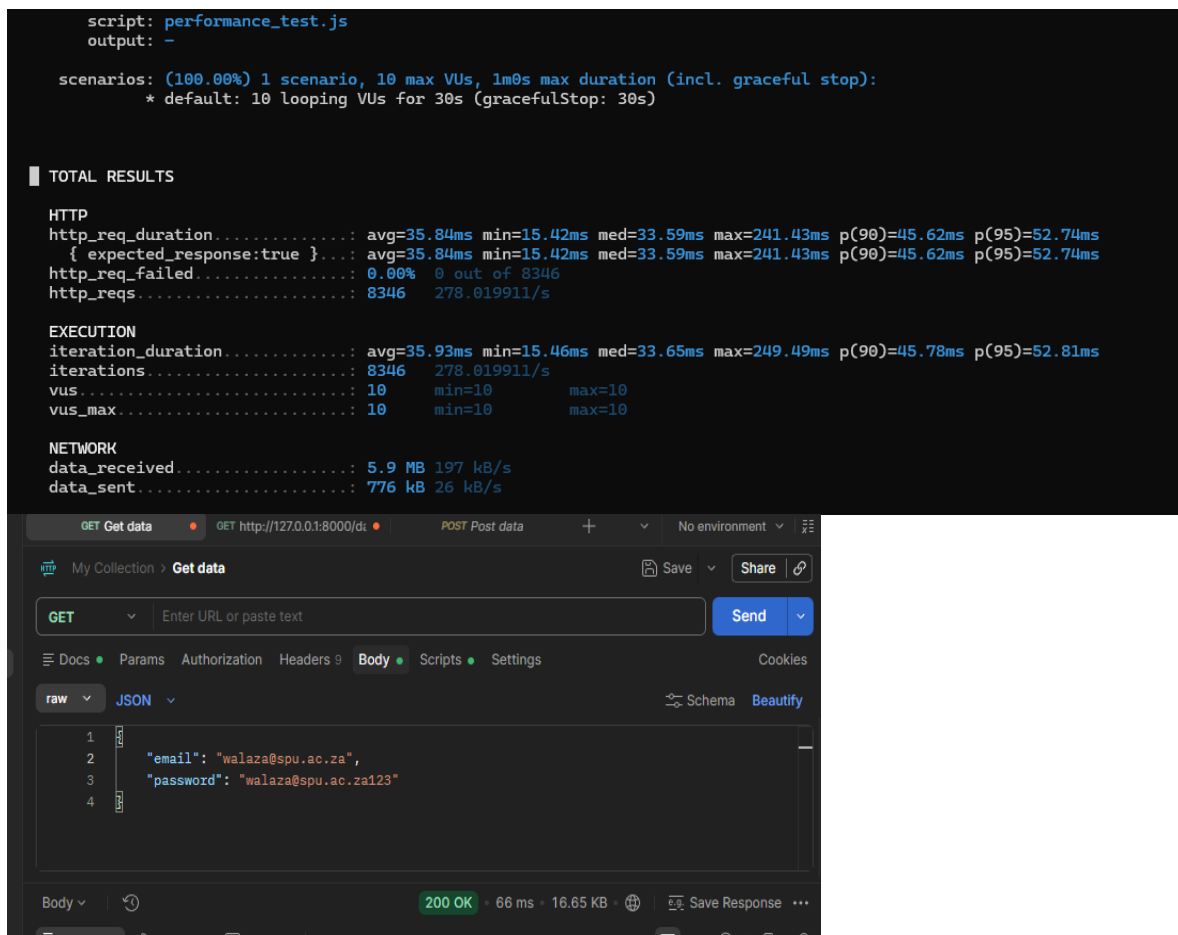
OK
Destroying test database for alias 'default'...

```

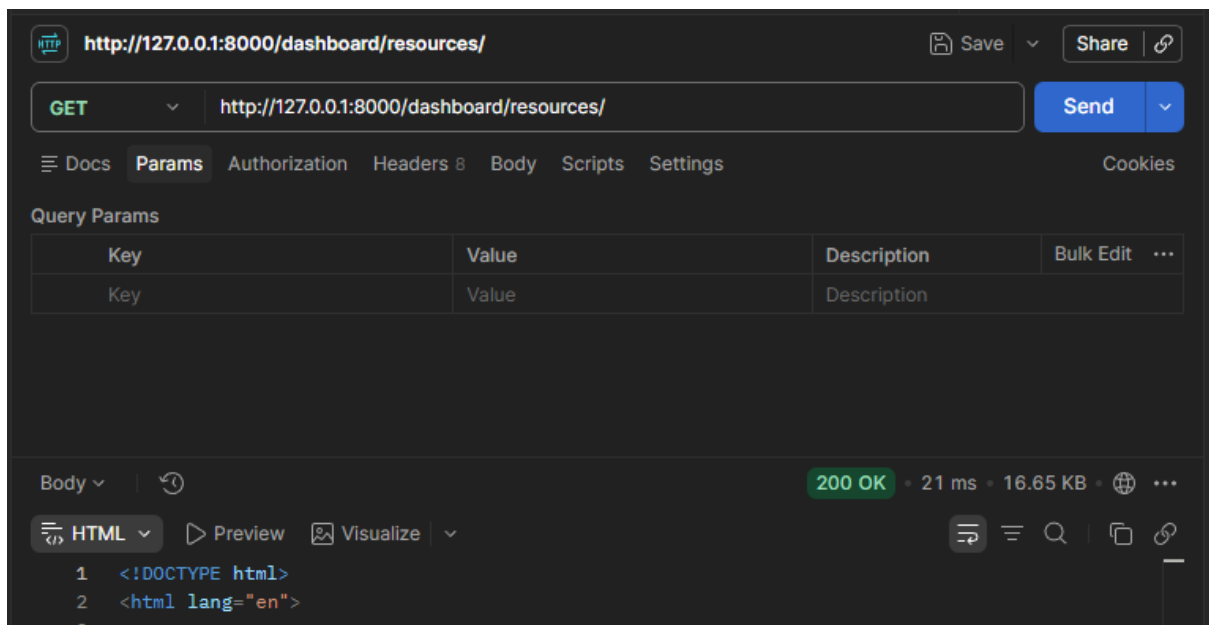
4.2 Unit Testing

Test Case	Objective	Type	Expected Result	Status
TC-01 User Registration	Register new User	Functional	User Account Created	Pass
TC-02 Login	Authenticate user	Functional	Valid token returned	Pass
TC-03 View Venues	Retrieve Venues list	Functional	Venues Displayed	Pass
TC-04 Create Booking	Created a valid booking	Functional	Booking Created	Pass
TC-05 Double Booking	Prevented overlapping booking	Functional	Conflict error(422)	Pass
TC-06 Cancel Booking	Cancel own booking	Functional	Booking Status cancelled	Pass
TC-07 Approve Booking	Admin Approves Booking	Functional	Status Updated	Pass
TC-08 Unauthorised Access	Access Without token	Security	401 Unauthorized	Pass

4.3 Performance and scalability notes



4.4 Api Testing



5. Report

5.1 Background and Context

The SPU Campus Booking System is designed to streamline the booking and management of campus venues, including lecture halls, laboratories, classrooms, tutorial rooms, event venues, auditoriums, and computer laboratories. The system replaces manual booking processes with a centralised digital platform that improves efficiency, reduces scheduling conflicts, and makes venue information and bookings easier to manage.

5.2 Methods

The system was developed using Django, MySQL, and RESTful architecture. The Agile methodology was followed throughout the development process. Testing was conducted using Pytest for unit testing, Postman for API testing, and k6 for performance testing.

5.3 Results and Discussion

The Campus Venue Booking System delivers a working service-driven solution addressing the problem statement in Section 1. Key outcomes:

- A functional REST API with 6 authentication endpoints and 12 resource/booking endpoints, returning consistent JSON and HTTP status codes
- Role-based access control enforced independently at the API layer and the web dashboard layer
- Conflict detection preventing double-booking of a venue for overlapping approved time slots
- A real-time availability endpoint calculating free and occupied slots for any requested date
- A token-based password reset flow
- 9 of 9 automated unit tests passing, with no Django system-check warnings

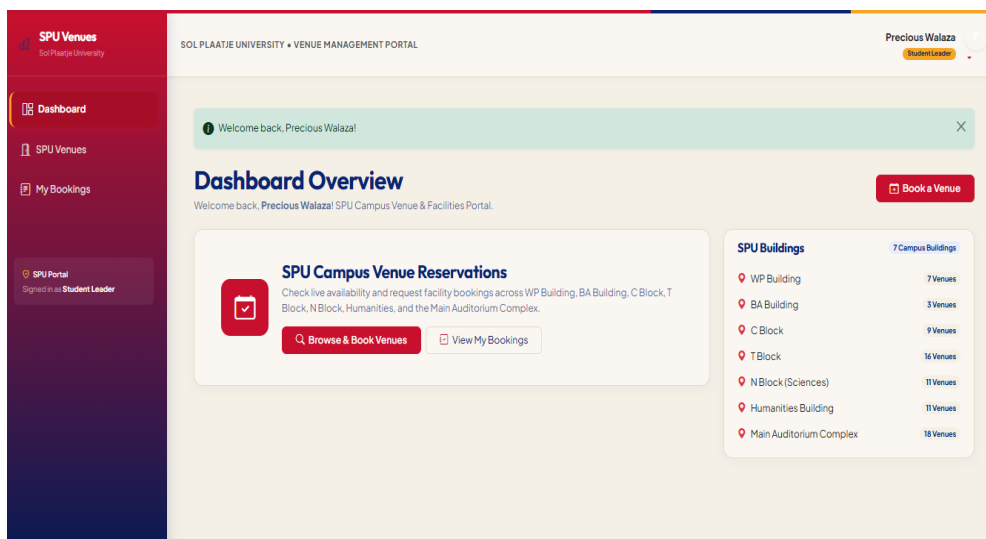
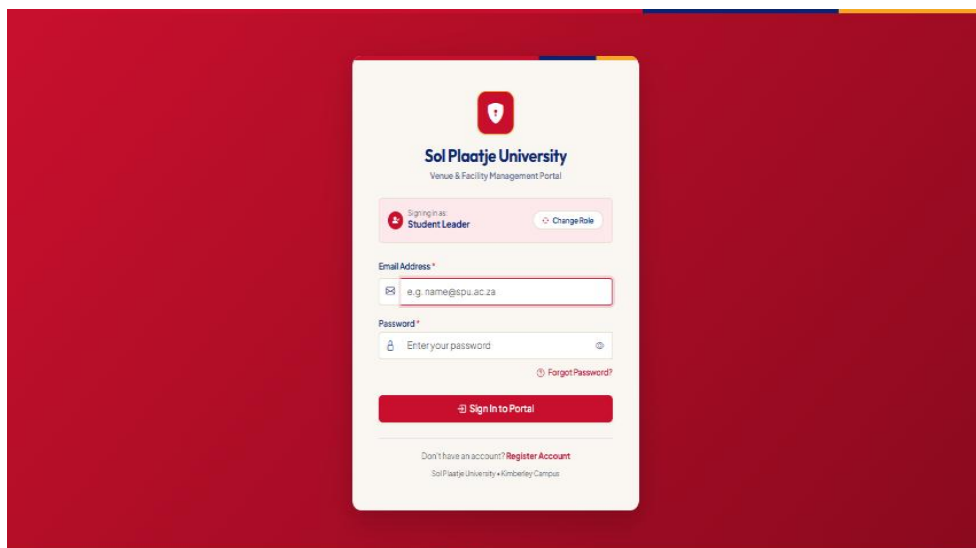
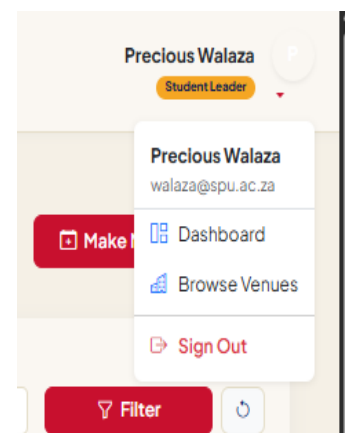
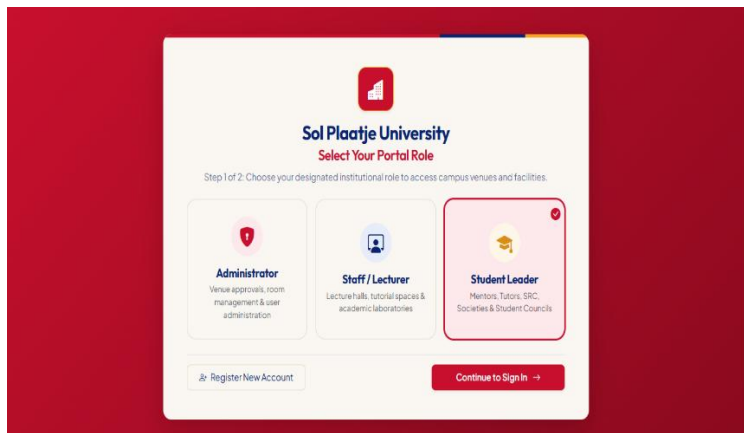
5.4 Limitations

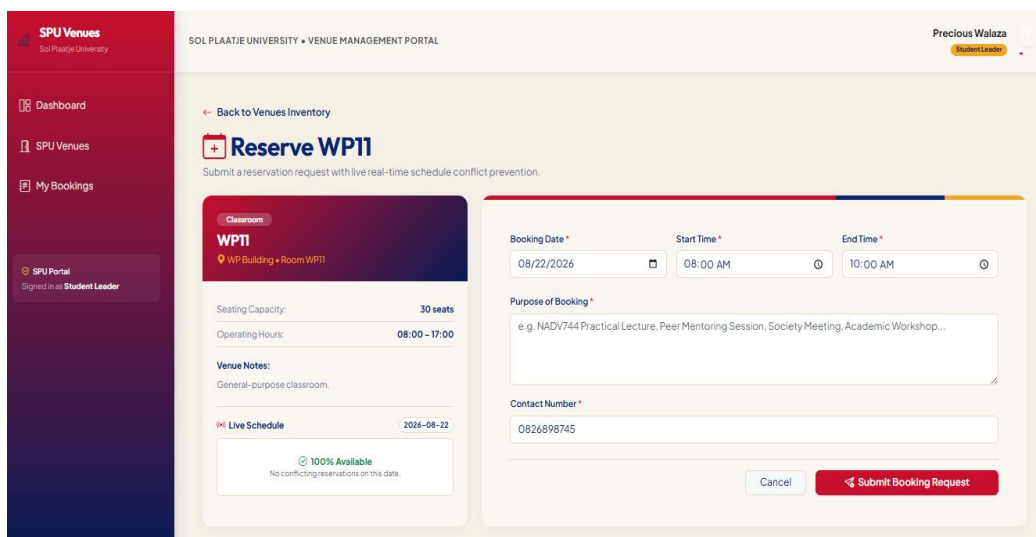
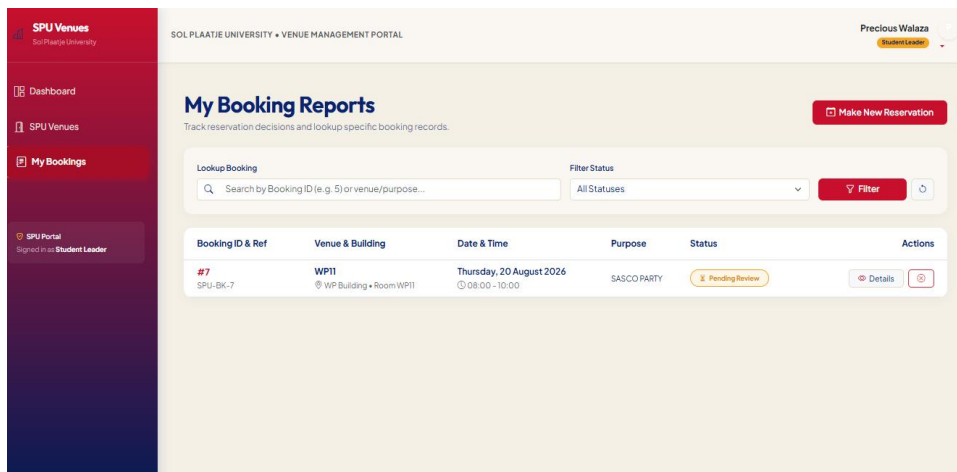
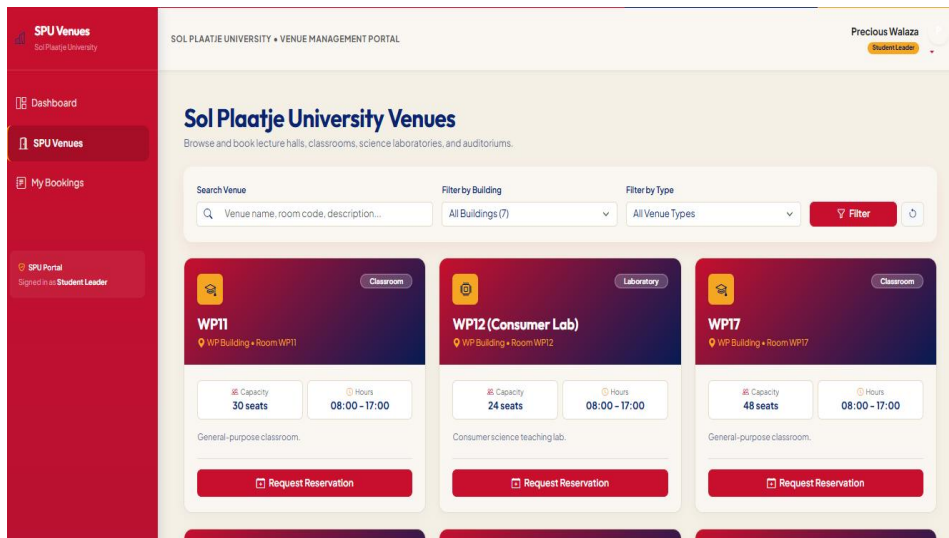
- No email or SMS notification is sent on booking approval or decline; status changes are visible only when the user checks their booking list
- Booking decisions do not currently include an optional administrator comment or reason field
- Performance testing has been conducted only in single-user local development mode; load testing under concurrent traffic is future work
- No calendar/timeline view for visualising bookings across a week or month

5.5 Future improvements

- Email notifications on booking status change
- Calendar/timeline view for administrators to see all venue schedules briefly
- Recurring booking support for regularly scheduled sessions
- Deployment to a cloud platform with PostgreSQL and HTTPS
- API rate limiting and audit logging
- Admin comment/reason field on approval or decline decisions

6. CODE & DEMO





6.1 How to run the System

Git Clone: <https://github.com/Maje107/ADVANCED-DEVELOPMENT-SOFTWARE-PROJECT>

Commands:

cd project directory

cd CODE

cd campus

python manage.py runserver

```
PS C:\Users\kamog\OneDrive - Sol Plaatje University\Desktop\School\4th Year\Semster 2\NADV744 ADVANCED DEVELOPMENT SYSTEMS\NADV Group Assignment\CODE\campus> python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...
```

7. Conclusion

The Campus Venue Booking System is a working, service-oriented Django application combining a documented REST API (Django REST Framework and JWT), role-based access control, server-side conflict detection, and an automated test suite of 9 passing tests. It replaces SPU's ad-hoc manual venue allocation with a centralised system where availability, approvals, and bookings are recorded and auditable. The layered separation between the API and the web dashboard, sharing one domain model, gives the system a clear path for the future enhancements listed in Section 5.3.